Universidad Autónoma del Estado de Hidalgo

Escuela Superior de Tlahuelilpan

Lenguajes de Programación

JOHN´S FRESH

Alumno:

San Agustín Velasco José María

Docente:

Mtra. Cornejo Velázquez Mónica

Tema:

Programación de Interfaz gráfica en Python con Tkinter

Grupo:

302

Semestre:

Tercero

Número de práctica:

8

Fecha:

26 de octubre de 2026

ÍNDICE

ÍNDICE 1
Introducción 2
Objetivo General 4
Objetivos específicos 4
Desarrollo de la práctica 5
Ejemplos de interfaces a seguir: 6
Imagen 1 6
Imagen 2 6
Imágenes del sistema de John's Fresh: 7
Imagen 1. Login 7
Imagen 2. Inicio 8
Imagen 3. Inventario 9
Imagen 4. Ventas 10
Imagen 5. Ventana de Ticket - Ventas 11
Imagen 6. Ventana de Nuevo Cliente - Ventas 12
Imagen 7. Clientes 13
Cuestionario final 14
Bibliografía 20
Anexos 21

Introducción

(López Ostenero & García Serrano, 2014, #)
Python es un lenguaje de programación de propósito general muy poderoso y flexible, a la vez que sencillo y fácil de aprender. Este lenguaje fue creado a principios de los noventa por Guido van Rossum en los Países Bajos. Es software libre, y está implementado en todas las plataformas y sistemas operativos habituales.

Toma características de lenguajes predecesores, incluso, compatibilizando la solución de varios de ellos. Por ejemplo, habilita tres formas de imprimir el valor de una variable: desde el entorno interactivo escribiendo su nombre (como en Basic), usando la función print, con concatenación de elementos (al estilo del write de Pascal) o bien con patrones de formato (al estilo del printf de C).

Python se desarrolla bajo una licencia de Open source o código abierto aprobada por OSI, por lo que se puede usar y distribuir libremente, incluso para uso comercial.
La Python Software Foundation (PSF) es una corporación sin fines de lucro 501 (c) (3) que posee los derechos de propiedad intelectual detrás del lenguaje de programación Python. Administramos las licencias de código abierto para Python versión 2.1 y posteriores.
El Python Package Index (PyPI) o en español significa Índice de paquetes de Python alberga miles de módulos de terceros para Python.
Tanto la biblioteca estándar de Python como los módulos aportados por la comunidad permiten infinitas posibilidades.

- Desarrollo web e Internet
- Acceso a la base de datos
- GUIs de escritorio
- Científico y numérico
- Educación
- Programación de red
- Desarrollo de Software y Juegos

Las características del lenguaje de programación Python se resumen a continuación:

- Es un lenguaje interpretado, no compilado, usa tipado dinámico, fuertemente tipado.
- Es multiplataforma, lo cual es ventajoso para hacer ejecutable su código fuente entre varios sistemas operativos.
- Es un lenguaje de programación multiparadigma, el cual soporta varios paradigmas de programación como orientación a objetos, estructurada, programación imperativa y, en menor medida, programación funcional.
- En Python, el formato del código (como lo es la indentación) es estructural.

En Python tiene varios tipos de datos compuestos estándar disponibles por defecto en el intérprete, como los tipos numéricos, secuencias, mapeos y conjuntos usados para agrupar otros valores.
Para el caso de las estructuras de datos se usan variables y constantes las cuales usan operadores para tratar los tipos de datos estándar.

En esta práctica de la asignatura Lenguajes de Programación se desarrollará **John 's Fresh**, un sistema de punto de venta para una pequeña tienda de abarrotes, utilizando Python y Tkinter, este último mencionado es una biblioteca que permite crear interfaces gráficas de usuario mediante ventanas, botones, campos de texto, etiquetas y otros elementos interactivos.

El propósito es diseñar interfaces gráficas que permitan al usuario iniciar sesión, consultar productos, seleccionar artículos para una compra, calcular los importes correspondientes y generar un ticket digital con el detalle de la venta, el subtotal, el impuesto y el total a pagar.

Asimismo, se busca aplicar los conocimientos de programación mediante el uso de variables, estructuras de datos, funciones y estructuras de control, con el fin de desarrollar una aplicación funcional y de fácil manejo. La información de los productos y las ventas se conservará mediante archivos locales, sin utilizar un sistema gestor de bases de datos.

De esta manera, la práctica permitirá relacionar los fundamentos del lenguaje Python con el desarrollo de interfaces gráficas y la solución de una necesidad cotidiana mediante un sistema informático.

Objetivo General

Desarrollar una aplicación de punto de venta para la tienda de abarrotes John’s Fresh mediante Python y la biblioteca Tkinter, implementando interfaces gráficas que permitan gestionar el acceso al sistema, consultar productos, registrar ventas y generar tickets digitales con el cálculo de los importes correspondientes.

Objetivos específicos

- Diseñar e implementar las interfaces gráficas necesarias para el inicio de sesión, el menú principal y el proceso de venta de la tienda John 's Fresh, utilizando la biblioteca Tkinter de Python.
- Implementar las funciones necesarias para consultar productos, seleccionar artículos, registrar cantidades y calcular el subtotal, los impuestos aplicables y el total de cada compra.
- Generar tickets digitales que presenten el detalle de los productos vendidos y los importes correspondientes, conservando la información de las ventas mediante archivos locales.



Desarrollo de la práctica

Desarrollar en Python, las interfaces necesarias para realizar un sistema de nombre “John’s Fresh” que se relacione con la actividad de una pequeña empresa que vende artículos a clientes.

- La empresa debe almacenar información de cada uno de sus clientes
- La empresa debe almacenar información de los artículos que vende
- El negocio debe almacenar datos de las ventas que realizan

Toda venta genera un ticket o nota de venta con su correspondiente detalle que describe:

- Producto(s) que se vende
- Cantidad
- El precio unitario del producto
- Importe del producto (Cantidad * Precio)

(Similares a los que te dan impresos en las tiendas, solo que este será sólo digital)

Al final del ticket, incluir:
subtotal = Sumatoria del importe del producto
iva = subtotal * 0.16
total = subtotal + iva

El sistema debe incluir un inicio de sesión con la identificación de los diferentes roles necesarios para la actividad.

Ejemplos de interfaces a seguir:

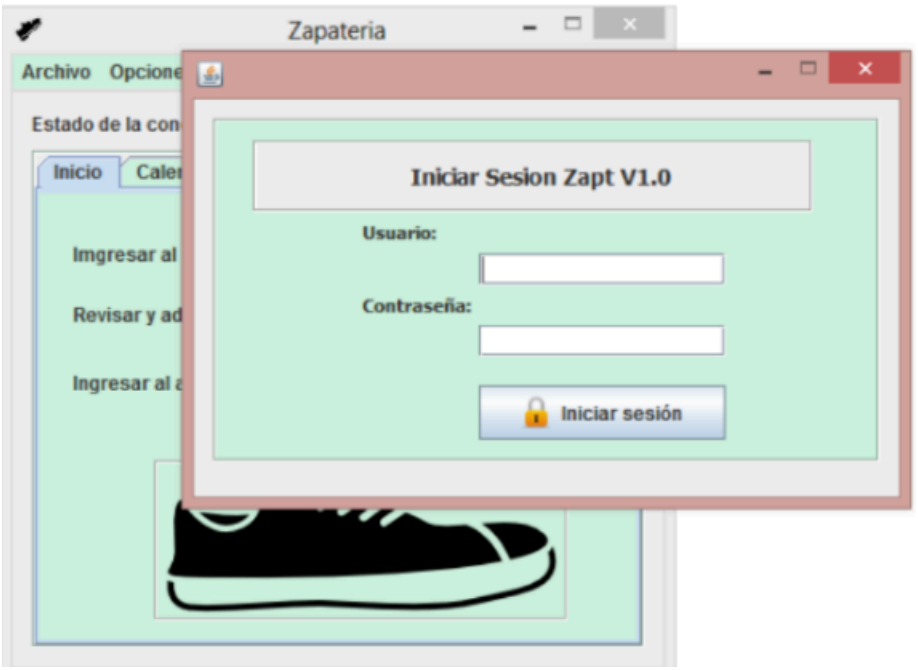


Imagen 1.

Pantalla de inicio de sesión y ventana principal del sistema

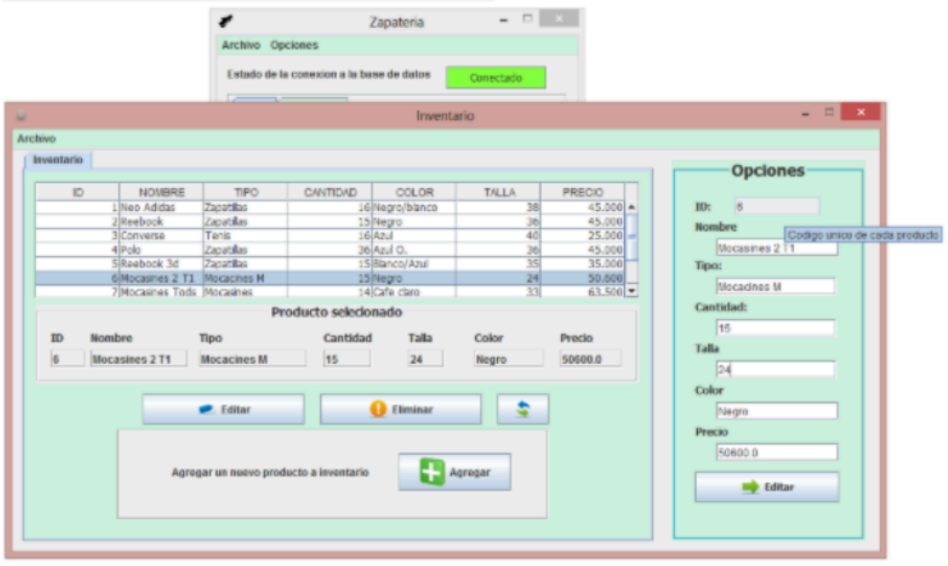


Imagen 2.

Elegir un código de producto que esté contenido en la tabla de datos de productos

A partir de estas imágenes, se realizará el sistema en Python con la librería TKinter, las imágenes a continuación muestran los avances:

Imágenes del sistema de John’s Fresh:



Imagen 1. Login

Ventana de “Login”, el usuario deberá ingresar su usuario y contraseña de forma correcta para poder entrar al sistema, así se abrirá la pantalla principal del sistema.

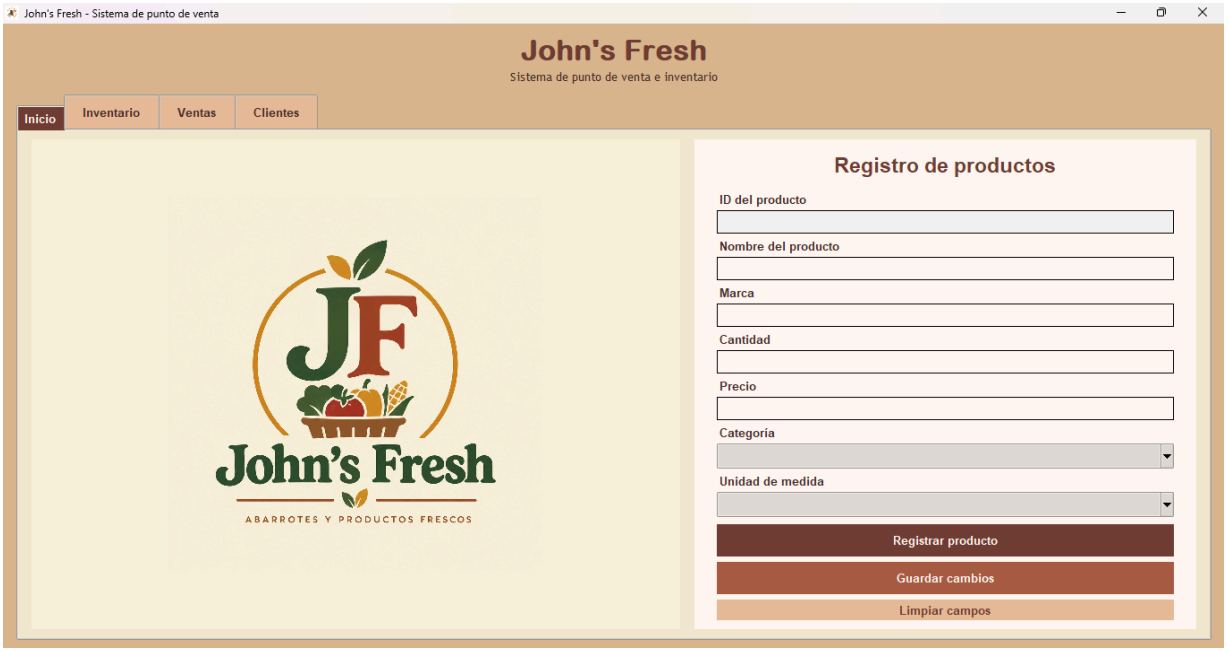


Imagen 2. Inicio

Ventana principal con la pestaña "Inicio", en esta pestaña se observa el logo de la tienda como su nombre, aquí el usuario puede ingresar nuevos productos a la tienda de acuerdo a los bloques que se muestran en el lado derecho de la pantalla, estos nuevos productos entran al sistema y se encuentran en la siguiente pestaña "Inventario", debe ingresar correctamente los datos para que sean válidos, presionar el botón "Registrar producto" para que se muestre en el inventario.



Imagen 3. Inventario

En la pestaña "Inventario" se muestra una tabla de productos ya realizados, en esta misma tabla se pueden agregar nuevos productos desde el Inicio, además de dos botones ya sea para eliminar algún producto o querer modificarlo, para modificarlo se hace en la pestaña "Inicio" y se presiona en el botón "Guardar cambios", al finalizar una venta, se eliminan las cantidades totales de cada producto, se es que se ha seleccionado.



Imagen 4. Ventas

En la pestaña "Ventas" se escogen los productos que se quieran comprar en la barra "Producto disponible", se ingresa el total que se quiera comprar y se presiona el botón "Agregar Venta", para cada producto se debe escoger a un cliente, ya sea registrandolo en el botón "Nuevo Cliente" o escogiendo uno ya existente en "Cliente de esta venta", se pueden quitar productos con el botón "Quitar producto", para poder finalizar la venta se debe poner la cantidad de dinero con la que se pagará en el lado izquierdo inferior, en el lado derecho inferior se muestra la cantidad que se debe pagar, con todo e IVA, ahora si se finaliza la venta en "Finalizar venta".



Imagen 5. Ventana de Ticket - Ventas

Al finalizar la venta, se muestra una ventana en donde imprime el ticket de la venta, esta misma ventana tiene un botón "Guardar ticket .txt" para poder guardar el ticket como un tipo de texto (.txt), así se puede compartir de una mejor manera a algún dispositivo de impresión o compartir por red.



Imagen 6. Ventana de Nuevo Cliente - Ventas

Si se quiere registrar a un nuevo cliente desde la pestaña de "Ventas"se muestra esta pequeña ventana para poder ingresar datos de un nuevo cliente muy rápidamente

El cliente se guarda con "Guardar Cliente" o "Cancelar" para cerrar la ventana sin guardar nada.



Imagen 7. Clientes

En la pestaña "Clientes" se muestra una tabla con algunos clientes ya registrados en el sistema, o también se puede ingresar a uno nuevo rellenando cada campo que se pide, así funciona el botón "Registrar Cliente", "Guardar cambios" permite guardar cualquier cambio que se le haya hecho a un cliente, y si queremos eliminar a algún cliente, solo se presiona el "Eliminar cliente", "Limpiar campos" vacía todos los datos que se hayan llenado en los campos de texto.

Y eso es todo el funcionamiento del sistema, para una mejor visualización, se muestra el siguiente vídeo del sistema:

https://drive.google.com/file/d/1qMA8WR4hKZ-C15B4peOu0v5U9FJD94Lm/view?usp=sharing

Cuestionario final

1. **¿Qué paradigmas de programación utiliza Python?**

(rootstack, 2023), (geeksforgeeks, 2020)

Python es multiparadigma, así que es capaz de soportar varios paradigmas, algunos de ellos son:

- ***Paradigmas de programación orientada a objetos:*** crear objetos, instanciar clases y hacer que cada elemento tenga un estado y comportamiento.
- ***Paradigmas de programación orientada a procedimientos:*** las series de los pasos computacionales se dividen en módulos, lo que hace que el código se agrupe en funciones y se ejecuta secuencialmente paso a paso.
- ***Paradigmas de programación funcional:*** están vinculados a las funciones matemáticas puras, utiliza declaraciones en lugar de sentencias.
- ***Paradigma de programación imperativa:*** describe cómo el código debe realizarse paso a paso mediante una secuencia detallada de instrucciones.

2. **¿ Cuál es la clasificación de los tipos de datos estándar en Python?**

(Pozo Ramos, 2026)

Hay 6 tipos de datos básicos en Python:

- ***int:*** números enteros
- ***float:*** números de punto flotante o decimales
- ***complex:*** números complejos
- ***str:*** cadenas y carácteres
- ***bytes, bytearray:*** bytes
- ***bool:*** valores booleanos (verdadero o falso)

3. **¿Cuáles son las palabras reservadas en Python? (Escribe un listado y una breve descripción):**

(El libro de Python, 2020)

- **if** (condicional): significa “si” y es una condicional por si ocurre un evento durante la compilación

- **elif** (condicional): significa “entonces si” y se usa por si no ocurre la primera condición, puede colocar otra
- **else** (condicional): significa “entonces”, sí la primera condición no se cumple, puede poner una excepción
- **while** (bucle): significa “mientras” y crea un bucle si su condición aún sigue funcionando
- **for** (bucle): significa “para” y crea un bucle de acuerdo a una cantidad seleccionada
- **break** (bucle): significa “romper” y rompe la ejecución del bucle
- **continue** (bucle): significa “continuar” y salta hasta el final del bloque
- **False** (valores): significa “falso” si no acierta el código
- **True** (valores): significa “verdadero” si todo está correcto
- **None** (valores): significa “nada” y se devuelve por defecto si no cuenta con un return
- **and** (operador lógico): significa “y” y sirve para agregar una condicional sobre otra
- **or** (operador lógico): significa “o” y sirve para comparar dos condicionales
- **not** (operador lógico): significa “no” y sirve para dar algo por no hecho
- **def** (funciones): nos permite crear una función
- **return** (funciones): nos devuelve uno o varios valores de la función
- **lambda** (funciones): nos permite crear una función “vaga” que no tiene un nombre, sino se asigna
- **pass** (funciones): dejar la función en blanco, no definirla
- **yield** (funciones): asociado a los generadores y las corrutinas
- **class** (clases): nos permite crear “clases” como en Java
- **assert** (excepciones): sirve para comprobar si una condición es verdadera o falsa
- **try** (excepciones): sirve para marcar un bloque de código si es que genera un error o excepción durante la ejecución
- **except** (excepciones): es un código alternativo después del try por si ocurre un error
- **finally** (excepciones): sirve para borrar y limpiar el error
- **raise** (excepciones): sirve para lanzar o generar una excepción de manera explícita durante la ejecución de un programa
- **global** (variable): nos permite tener una variable que es usada por todas las clases y funciones
- **nonlocal** (variable): es útil cuando se tiene funciones anidadas
- **from** (módulos): nos permite llamar librerías o módulos internas o externas de Python

- **import** (módulos): nos permite importar una librería o módulo de Python
- **in** (pertenencia): nos permite saber si un determinado elemento está en un clase iterable
- **is** (pertenencia): nos permite saber si dos variables apuntan en realidad al mismo objeto
- **del** (eliminar variables): nos permite eliminar una variable del *scope,* cuando hay grandes datos
- **with** (context managers): sirve para gestionar recursos de forma automática
- **as** (context managers): sirve para crear un nombre o alias al importar módulos o al manejar excepciones
- **async** (concurrencia): sirve para escribir código asíncrono y no bloqueante
- **await** (concurrencia): se utiliza dentro del async para "esperar" el resultado de una operación lenta

4. ¿Qué operadores de asignación se pueden utilizar en Python?

(El libro de Python, 2020)

Los operadores de asignación en Python permiten realizar una operación y almacenar su resultado en la variable inicial:

- = (igual): asigna una variable de la izquierda el contenido de la derecha
- += (más igual): acorta esta expresión "x = x + 1"
- **-=** (menor igual): acorta esta expresión "x = x - 1"
- ***=** (por igual): acorta esta expresión "x = x * 2"
- /= (entre igual): acorta esta expresión "x = x / 2"
- **%=** : módulo de la adición de dos variables y almacenar el resultado de la primera
- //= : realiza la operación cociente entre dos variables y almacena el resultado en la primera "x = x // 2"
- ****=** : realiza la operación exponente del primer número elevado al segundo y almacena el resultado en la primera "x = x * * 2"
- **&=** : realiza la comparación entre bit y bit de dos variables y almacena el resultado en la primera "x = x & 1"
- |= : realiza el operador | elemento a elemento entre dos variables y almacena su resultado a la primera "x = x | 2"

- **^=** (potencia igual): realiza la potencia y acorta la expresión “x = x ^ 2”
- **>>=** : es similar al >> pero permite almacenar el resultado en la primera variable “x = x >> 3”
- **<<=** : es similar al << pero permite almacenar el resultado en la primera variable “x = x << 1”

5. ¿Cómo se realizan los comentarios de línea y multilínea en Python?

(geeksforgeeks, 2019)

Los comentarios de una sola línea en Python se hacen con el gato o hashtag **(#).**

Los multilínea tiene dos formas, colocar varios gatos o colocar triples comillas dobles o simples:

```
#          “””          ‘’’          Estos son ejemplos de
#          “””          ‘’’          cómo colocar comentarios
#          “””          ‘’’          multilínea en Python
```

6. ¿Qué es una Lista en Python?, y describe los métodos que utiliza para la lista:

(El libro de Python, 2020)

Las listas en Python son un tipo de dato que permite almacenar datos de cualquier tipo. Son mutables y dinámicas, un tipo de estructura de datos más versátiles del lenguaje, ya que permite almacenar un conjunto arbitrario de datos.

Los métodos que usa son:

- **append()** → añade un elemento al final de la lista → append(<obj>)
- **extend()** → permite añadir una lista a la lista inicial → extend(<iterable>)
- **insert()** → añade un elemento en una posición o índice determinado → insert(<index>, <obj>)
- **remove()** → recibe como argumento un objeto y lo borra de la lista → remove(<obj>)
- **pop()** → elimina por defecto el último elemento de la lista, pero si se pasa como parámetro un *índice* permite borrar elementos diferentes del último → pop(index = -1)
- **reverse()** → invierte el órden de la lista

- **sort()** → ordena los elementos de menor a mayor por defecto o de mayor a menor si se usa el *reverse = True*
- **index()** → recibe como parámetro un objeto y devuelve el índice de su primera aparición. También permite introducir un parámetro opcional que representa el índice desde el que comenzar la búsqueda del objeto → index(<obj>[, index])

7. ¿Qué es una Tupla y cuáles son los métodos que se utilizan en Python para las tuplas?

(El libro de Python, 2020)

Son un tipo o estructura de datos que permite almacenar datos de una manera muy parecida a las listas, con la salvedad de que son inmutables (no pueden ser declaradas ni modificadas), al igual de que pueden ser más rápidas.

Los métodos que usa son:

- **count()** → cuenta el número de veces que el objeto pasado como parámetro se ha encontrado en la lista → count(<obj>)
- **index()** → busca el objeto que se la pasa como parámetro y devuelve el índice en el que se ha encontrado, en caso de no encontrarse, devuelve un *ValueError*. El método index() también acepta un segundo parámetro opcional, que indica a partir de qué índole empezar a buscar el objeto → index(<obj>[, index])

8. ¿Qué son los Conjuntos en Python y cuáles son los métodos que se utilizan para los conjuntos?

(El libro de Python, 2020)

Los conjuntos (o también llamados **set**) son uns estructura de datos usada para almacenar elementos de una manera similar a las listas, pero con ciertas diferencias:

- Los elementos de un set son **único,** lo que significa que no pueden haber elementos duplicados
- Los set son **desordenados,** lo que significa que no mantienen el orden de cuando son declarados
- Sus elementos deben ser **inmutables**

Los métodos que usa son:

- **add()** → permite añadir un elemento al set → s.add(<elem>)
- **remove()** → elimina el elemento que se pasa como parámetro. Si no se encuentra, marcará como *KeyError* → s.remove(<elem>)
- **discard()** → es muy parecido al remove(), borra el elemento que se pasa como parámetro, y si no se encuentra no hace nada → s.discard(<elem>)
- **pop()** → elimina un elemento aleatorio del set → s.pop()
- **clear()** → elimina todos los elementos del set → s.clear()

Y hay más elementos que permiten realizar operaciones con dos o más, como la **unión** o **intersección**.

9. **¿Qué estructuras de control utilizaste para el desarrollo de la práctica?**

Las principales estructuras de control que utilicé fueron ***if, elif*** y ***else*** para tomar decisiones dentro del programa, por ejemplo, al validar el inicio de sesión, comprobar los datos ingresados y verificar si se cumplen las condiciones para realizar una venta. También utilicé estructuras repetitivas, como ***for*** para recorrer listas de productos o clientes cuando es necesario. Además, utilicé ***try*** y ***except*** para manejar posibles errores, como cuando no se puede cargar una imagen. Estas estructuras permiten que el programa siga un orden lógico y responda adecuadamente a las acciones del usuario.

10. **Si utilizaste alguna de las funciones de Python, describe el procedimiento realizado para utilizarla:**

Sí, utilicé funciones de Python para organizar el código y realizar diferentes tareas dentro del sistema. Por ejemplo, creé funciones para registrar y consultar clientes, administrar productos, realizar ventas, calcular el subtotal, el IVA del 16 % y el total de la compra, así como generar y guardar los tickets de venta. También utilicé funciones de Tkinter para crear ventanas, botones, etiquetas, tablas y campos de texto. El procedimiento consistió en definir cada función con ***def,*** escribir las instrucciones que debía ejecutar y llamarla cuando el usuario realizaba una acción, como presionar un botón. De esta manera, el código queda más organizado y es más fácil de mantener.

Bibliografía

El libro de Python. (2020). Palabras reservadas en Python. Retrieved Octubre 19, 2026, from https://ellibrodepython.com/palabras-reservadas-python#valores-false-true-none

El libro de Python. (2020). Operadores de asignación. Retrieved Octubre 19, 2026, from https://ellibrodepython.com/operadores-asignacion#operador-

El libro de Python. (2020). Listas en Python. Retrieved Octubre 19, 2026, from https://ellibrodepython.com/listas-en-python#m%C3%A9todos-listas

El libro de Python. (2020). Tupla (tuple). Retrieved Octubre 19, 2026, from https://ellibrodepython.com/tuplas-python#m%C3%A9todos-tuplas

El libro de Python. (2020). Set. Retrieved Octubre 19, 2026, from https://ellibrodepython.com/sets-python#m%C3%A9todos-sets

geeksforgeeks. (2019, Diciembre 14). Multiline Comments in Python. Retrieved Octubre 19, 2026, from https://www.geeksforgeeks.org/python/multiline-comments-in-python/

geeksforgeeks. (2020, Marzo 11). Programming Paradigms in Python. Retrieved Octubre 19, 2026, from https://www.geeksforgeeks.org/python/programming-paradigms-in-python/

J., L., & G., C. (2014). Programación en Python - Nivel Básico. https://entrenamiento-python-basico.readthedocs.io/es/latest/index.html

López Ostenero, F., & García Serrano, A. M. (2014). *Teoría de los lenguajes de programación*. Editorial Universitaria Ramón Areces.

Pozo Ramos, L. (2026, Octubre 03). Basic Data Types in Python: A Quick Exploration. Retrieved Octubre 19, 2026, from https://realpython.com/python-data-types/

rootstack. (2023, Diciembre 05). Paradigmas soportados en Python. Retrieved Octubre 19, 2026, from https://rootstack.com/es/blog/paradigmas-soportados-en-python

Anexos

1. Página principal donde se sacaron las funciones:

https://docs.python.org/3/library/tkinter.html

2. Canal oficial y lista de reproducción de vídeos del youtuber ***pildorasinformaticas:***

https://www.youtube.com/@pildorasinformaticas/featured

- Videos del 42 al 54: https://www.youtube.com/watch?v=tDYr14IIu_4&list=PLU8oAlHdN5BlvPxziopYZRd55pdqFwkeS

3. Repositorio de GitHub donde se subieron los cambios:

https://github.com/Jose-18-debug/John-s-Fresh.git